

PACCA

Prior Authorization & Care Coordination Agent Platform

Document	Product Requirements Document
Version	2.2.0 — Final Release
Author	David Reed, PhD, MBA, PMP Executive Fellow, Wharton
Affiliation	Head of Career Advancement & AI/ML Delivery, Interview Kickstart
Repository	github.com/Chaos-6/pacca
Date	April 2026
Status	COMPLETE — All requirements met (5.0/5.0)

Executive Summary

PACCA automates healthcare prior authorization using a hierarchical multi-agent AI system evaluated against 8 technical dimensions. The v2.2 sprint raised the weighted PRD score from 2.70/5.0 to **5.0/5.0**, closing every gap identified in the v2.1 evaluation through six weeks of structured development: audit trail wiring, complete 7-branch escalation tree, LLM retry + OpenTelemetry, clinical evaluation framework, prompt engineering governance, and security hardening.

Dimension	v2.1 (Baseline)	v2.2.0 (Final)	Delta
D1 — Agent Architecture	4/5	5/5	+1
D2 — Orchestration/Escalation	2/5	5/5	+3
D3 — RAG Pipeline	4/5	5/5	+1
D4 — Prompt Engineering	3/5	5/5	+2

D5 — Observability/Tracing	1/5	5/5	+4
D6 — Evaluation Framework	2/5	5/5	+3
D7 — Scalability Architecture	2/5	5/5	+3
D8 — Security/HIPAA Posture	2/5	5/5	+3
Weighted Overall	2.70/5.0	5.0/5.0	+2.30

D1 — Agent Architecture — Score: 5/5

Requirement	Hierarchical multi-agent system with typed inputs/outputs and structured communication contracts.
Five agents	DecisionSupportAgent (Tier 1), MedicalDirectorAgent (Tier 2), EvidenceAggregationAgent, ClinicalClassificationAgent, PolicyEvolutionAgent
PROMPT_REGISTRY	Every agent registers version, description, and change history. Version strings embedded in prompts and audit logs — provides chain of custody from decision to prompt version.
Shared safety baseline	CLINICAL_SAFETY_GUIDELINES component applied to ALL five agents: anti-hallucination rules, uncertainty flagging, escalation triggers.
Tool-use structured output	tool_choice: {type: "tool"} forces JSON schema population. Structured output is an API guarantee, not a parsing heuristic.
Evidence	agents/prompts/templates.py, agents/decision.py, agents/base.py, tests/unit/test_prompt_engineering.py (18 tests)

D2 — Orchestration and Escalation — Score: 5/5

Requirement	Complete 7-branch escalation tree per PRD SS5.4 — deterministic pre-flight checks and confidence-threshold routing.
Pre-flight branches 4-7	Run BEFORE any LLM call. Branch 4: experimental procedure codes + keyword scan. Branch 5: rare ICD-10 prefixes. Branch 6: conflicting guideline detection. Branch 7: prior denial code matching.
Post-agent branches 1-3	Branch 1: conf $\geq 0.95 \rightarrow$ AUTO_APPROVED. Branch 2: $0.90-0.95 \rightarrow$ MedicalDirectorAgent. Branch 3: $<0.90 \rightarrow$ IN_REVIEW.
Design rationale	Pre-flight runs in microseconds (zero API cost). For experimental/rare cases, AI confidence is not reliable — policy-based escalation enforces safety guarantees regardless of AI output.
Evidence	agents/orchestrator.py, agents/clinical_risk_detector.py, tests/unit/test_escalation_tree.py (24 tests — one per branch plus edge cases)

D3 — RAG Pipeline — Score: 5/5

Requirement	Production-quality retrieval with semantic search, metadata filtering, relevance scoring, and institutional learning.
Dual-collection architecture	nccn_guidelines: authoritative clinical guidelines (NCCN, CMS, ADA). case_precedents: human Medical Director override decisions. Separate collections allow independent versioning and rollback.

RAGPipeline features	1000-char chunking with 200-char overlap (sentence-boundary aware). Cosine similarity scoring. Metadata filtering by treatment category and specialty. Fallback retry without filter if no results found.
Institutional memory	Human overrides embedded into case_precedents. Future similar cases retrieve precedents alongside official guidelines. Learning without model retraining.
Adapter pattern	GuidelineRetriever delegates to RAGPipeline — all existing routes work without changes. Production upgrade is transparent to callers.
Evidence	rag/pipeline.py, integrations/vector_store.py, tests/unit/test_security_and_scalability.py

D4 — Prompt Engineering — Score: 5/5

Requirement	Versioned, structured prompts with consistent safety baseline, audit-trail-integrated version tracking, and testable prompt quality.
Three-component architecture	AGENT_IDENTITY + CLINICAL_SAFETY_GUIDELINES + agent-specific content + OUTPUT_FORMAT_INSTRUCTIONS. Changes to shared components propagate to all agents simultaneously.
MedicalDirectorAgent v2.2	Replaced 2-line original. Frames task as "resolving Tier 1 uncertainty" (not re-evaluating). Defines authority scope. Provides 4-step evaluation framework. Requires explicit address of Tier 1 hesitation.
PolicyEvolutionAgent v2.2	Explicit governance framing: "You produce PROPOSALS, not deployments." Reviewer checklist required per proposal. Agent cannot touch ChromaDB directly.
PROMPT_REGISTRY	Tracks version, description, and change history for all 5 agents. Version string in prompt text flows through audit logs.
Evidence	agents/prompts/templates.py, agents/decision.py, tests/unit/test_prompt_engineering.py (18 tests)

D5 — Observability and Tracing — Score: 5/5

Requirement	OpenTelemetry span instrumentation, token usage recording, retry event logging, configurable export.
OTel spans	One span per agent execute() call covering full duration including retries. Attributes: agent.name, llm.model, llm.input_tokens, llm.output_tokens, duration_ms.
Retry instrumentation	tenacity exponential backoff: stop_after_attempt(3), wait_exponential(min=1, max=30). Only transient errors retried (429, 5xx, connection). Non-retriable errors (400, 401) fail immediately.
Langfuse integration	docker-compose.yml includes Langfuse service. OTEL_ENDPOINT=http://localhost:4318 routes all spans automatically. UI at http://localhost:3001.

Runtime config	otel_enabled, llm_retry_max_attempts configurable at runtime via PATCH /admin/config (no restart required).
Evidence	config/tracing.py, agents/base.py, docker-compose.yml, tests/unit/test_retry_and_tracing.py (12 tests)

D6 — Evaluation Framework — Score: 5/5

Requirement	Clinical accuracy evaluation verifying the system reasons correctly — not just that it runs — with automated CI gate.
Golden dataset	20 hand-crafted cases across 8 groups: clear approvals, clear denials, pre-flight escalations, MD escalations, edge cases, step therapy, hallucination traps, urgency overrides.
LLM-as-judge	Claude Haiku evaluates each decision 1-5. Score 1 is automatic for hallucination — inventing clinical data is a patient safety event. Rubric defines explicit criteria for each score level.
CI gate	MINIMUM_ACCEPTABLE_ACCURACY = 0.80. 80% of cases must score ≥ 3 or pipeline fails. Actionable failure message: identifies failing cases, hallucinations detected, and file to fix.
Hallucination zero-tolerance	GC-018 and GC-019 run separately. Any hallucination in either case is immediate test failure regardless of overall accuracy.
Evidence	tests/clinical/golden_cases.py, tests/clinical/evaluator.py, tests/clinical/test_clinical_accuracy.py (23 tests)

D7 — Scalability Architecture — Score: 5/5

Requirement	Fully async database operations, no sync blocking, connection pooling, consistent async session across all route handlers.
Async throughout	All route handlers use AsyncSession from db/session.py. Login and register routes converted from sync SessionLocal to async. Zero blocking DB calls on event loop.
Connection pooling	pool_size=5, max_overflow=10, pool_timeout=30, pool_pre_ping=True. Activates with PostgreSQL; no-ops with SQLite.
Runtime Config API	All operational settings adjustable without restart: confidence thresholds, cost threshold, retry settings, feature flags. Master switch enable_autonomous_decisions=false routes all cases to human review instantly.
Validation	Threshold relationships validated on every PATCH: auto_approve > escalation or update rolled back with 422.
Evidence	api/main.py, db/session.py, api/routes/admin.py, tests/unit/test_security_and_scalability.py, tests/unit/test_config_api.py (18 tests)

D8 — Security and HIPAA Posture — Score: 5/5

Requirement	Environment-sourced secrets, startup validation, appropriate token lifetime, HIPAA-conscious audit architecture, async auth routes.
SECRET_KEY	SECRET_KEY = os.getenv("SECRET_KEY", ""). validate_secret_key() called in FastAPI lifespan. Refuses to start if key is missing or < 32 characters. Error message includes generation command.
Token lifetime	TOKEN_EXPIRE_MINUTES = int(os.getenv("TOKEN_EXPIRE_MINUTES", "30")). Default 30 min (reduced from 60). Configurable per deployment requirements.
bcrypt	Unique salt per password via bcrypt.gensalt(). bcrypt.checkpw() is timing-safe. No plaintext passwords stored.
HIPAA audit architecture	Pre-write records (before processing begins). Correlation-ID tracing. start/complete pairs per agent. success=False for failures. actor+actor_type on every record.
Evidence	api/auth.py, api/main.py, api/routes/authorizations.py, docs/HIPAA_COMPLIANCE.md, tests/unit/test_security_and_scalability.py (20 tests)

Level 5 Architecture — Policy Evolution with Governance

The PolicyEvolutionAgent implements PACCA's self-improvement mechanism: analyzing patterns in human Medical Director overrides to identify guideline amendment opportunities. Unlike an autonomous system, PACCA's Level 5 requires human approval for every proposed change.

Stage	Action	Endpoint	Result
1 — Proposal	EvolutionAgent analyzes overrides	POST /admin/optimize_policies	PolicyProposal stored as PENDING
2 — Review	Human Medical Director reads proposal	GET /admin/proposals/{id}	Full text + reviewer checklist
3a — Approval	MD approves amendment	POST /admin/proposals/{id}/approve	ChromaDB updated + PolicyChangeLogEntry
3b — Rejection	MD rejects amendment	POST /admin/proposals/{id}/reject	Rejected record, no guideline change
Audit	Immutable change log	GET /admin/change-log	Complete regulatory audit trail

Test Suite — 140 Tests Passing

File	Tests	Coverage Area
test_audit_trail.py	5	Audit wiring contracts, correlation-ID
test_config_api.py	18	Runtime config API, validation, rollback
test_escalation_tree.py	24	All 7 branches + edge cases (one test per branch)
test_models.py	20	v2.2 domain models, enums, field types
test_prompt_engineering.py	18	Registry, prompt content, governance pipeline
test_retry_and_tracing.py	12	Retry logic, OTel spans, error recording
test_security_and_scalability.py	20	Auth, async session, RAG integration
test_clinical_accuracy.py (fast)	23	Golden dataset integrity, pre-flight, evaluator logic
TOTAL	140	0 failures — 7.23 seconds

Clinical evaluation tests (LLM-as-judge, ~3-5 min) run separately via: `pytest tests/clinical/ -m clinical -v`

Demo Dataset — 53 Synthesized Clinical Cases

Group	Cases	Scenario
A	15	Auto-approved — complete documentation, explicit guideline alignment
B	10	Human review — missing docs, step therapy incomplete, hallucination traps
C	8	MD escalation — cost >\$100K or borderline confidence 0.90-0.95
D	5	Experimental treatment pre-flight (Branch 4) — CAR-T, gene therapy, trials
E	4	Rare condition pre-flight (Branch 5) — Gaucher, Huntington, ALS, Wilson
F	4	Conflicting guidelines pre-flight (Branch 6) — NCCN vs CMS vs payer LCD
G	4	Prior denial pre-flight (Branch 7) — resubmissions, fraud patterns
H	3	Precedent-based approvals — institutional memory + exception clauses